

AI Candidate-Ranking System

Intelligent Candidate Discovery & Ranking Challenge

Team kafka_consumer

github.com/ojas4414/IndiaRun

The LLM should understand. It should never decide.

The Problem

What we were asked to solve

- Rank the top 100 of 100,000 candidates for one Senior AI Engineer JD.
- "Most AI keywords wins" is an explicit trap built into the dataset.
- Traps: keyword stuffers, plain-language strong fits, behavioral twins, ~80 honeypots (impossible profiles).
- Constraints: ≤ 5 min, 16 GB RAM, CPU-only, no network at ranking; must be deterministic.

Thesis

The LLM should understand, never decide

- Letting an LLM rank is non-deterministic, slow, and can't run offline in budget.
- So we split UNDERSTANDING (offline embeddings) from RANKING (deterministic scoring).
- rank.py makes zero LLM/network calls and returns byte-identical output on identical input.

Pipeline: retrieve -> rerank -> disambiguate

Architecture

- Precompute (no time limit): honeypot filter -> sentence-transformer embeddings -> FAISS index.
- Stage 1 Retrieval: FAISS ANN -> top-2000 semantic funnel (recall).
- Stage 2 Feature rerank: trajectory-DP 35% | skill-graph 22% | behavioral 18% | attention 15% | semantic 10% - disqualifiers.
- Stage 3 Attention rerank over the top-600 survivors (precision + evidence).
- Stage 4 Behavioral-twin disambiguation -> top-100 + evidence-grounded reasoning (~40s CPU).

How each trap is handled

Trap defenses map directly to the brief

- Keyword stuffing -> skill-graph + trajectory carry 57% of weight; off-domain titles hard-disqualified.
- Plain-language Tier-5s -> semantic retrieval + sentence attention surface hidden production evidence.
- Behavioral twins -> cluster near-duplicates; the most-available twin leads, rest demoted.
- ~80 honeypots -> date/tenure impossibility checks at precompute (52 caught; 0 in top-100).
- Consulting-only / CV-speech-only / title-chasers / no-recent-code -> explicit disqualifier rules.

Novel #1: contrastive-facet sentence attention

Precision + explainability

- Cross-attention pooling: JD facets = queries, candidate sentences = keys/values (ColBERT-style MaxSim).
- Recovers a single strong sentence that document-level embedding would average away.
- Contrastive anti-fit facets cancel sentence-level keyword stuffing (e.g. "SEO articles that ranked in search").
- The highest-attention sentence is surfaced as concrete evidence in each candidate's reasoning.

Novel #2: twins + adversarial robustness

Things a cosine-similarity submission won't have

- Behavioral-twin disambiguation: paper-identical profiles are separated by availability signals.
- Adversarial test: inject every JD skill + fake 95/100 assessments into profiles that shouldn't rank.
- Off-domain promotion under attack: FULL system 0% vs naive keyword ranker 100%.
- We report an honestly-found residual weakness (generic engineers) + its mitigation. Rigor over spin.

We measured it (ablation + baselines)

Evaluation the JD explicitly asks for: NDCG / MRR / MAP

- NDCG@100: full 0.876 vs naive_keyword 0.615 vs semantic_only 0.508.
- Off-domain titles in top-100: 4% (full) vs 27% (semantic-only).
- Ablation: trajectory is the dominant fit signal; semantic is mainly a RECALL signal (near-inert in final ordering).
- Weights were tuned from this harness, not guessed. Caveat: rubric is weak supervision; label-free trap metrics are load-bearing.

Results & guarantees

Meets every hard constraint

- Ranks the full 100k in ~40s (limit: 5 min); 0 honeypots and 100% on-domain titles in the top-100.
- Deterministic: two runs produce byte-identical output (MD5-verified).
- Offline: embedding model baked into the Docker image; ranking sets `TRANSFORMERS_OFFLINE=1`.
- Reproducible: ``docker compose up --build``. 24 passing tests.